

# Слкуни Герман РТ5-61Б

## Лабораторная работа 6

---

### Задание

1. Выберите набор данных (датасет) для решения задачи классификации или регрессии.
2. В случае необходимости проведите удаление или заполнение пропусков и кодирование категориальных признаков.
3. С использованием метода `train_test_split` разделите выборку на обучающую и тестовую.
4. Обучите следующие ансамблевые модели:

одну из моделей группы стекинга.

модель многослойного персептрона. По желанию, вместо библиотеки `scikit-learn` возможно использование библиотек `TensorFlow`, `PyTorch` или других аналогичных библиотек.

двумя методами на выбор из семейства МГУА (один из линейных методов COMBI / MULTI + один из нелинейных методов MIA / RIA) с использованием библиотеки `gmdh`.

В настоящее время библиотека МГУА не позволяет решать задачу классификации !!! Оцените качество моделей с помощью одной из подходящих для задачи метрик. Сравните качество полученных моделей.

### Описание датасета

Набор данных `breast_cancer` (Wisconsin Diagnostic Breast Cancer) содержит:

Объекты: 569 образцов опухолевых клеток.

Признаки (30 числовых):

Для каждого из 10 измеряемых свойств (`radius`, `texture`, `perimeter`, `area`, `smoothness`, `compactness`, `concavity`, `concave points`, `symmetry`, `fractal dimension`) вычислены три статистики: `mean` (среднее по клеткам), `se`

(стандартная ошибка), worst (наихудшее значение). Целевая переменная (target): - бинарная, где 0 = malignant (злокачественная) 1 = benign (доброкачественная)

```
In [1]: from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split

data = load_breast_cancer()
X_train, X_test, y_train, y_test = train_test_split(
    data.data, data.target, test_size=0.3, random_state=42, stratify=data.target
)

print("Features (30):", data.feature_names)
print("Target names:", data.target_names)
print("X shape:", data.data.shape)
print("y distribution:",
      {name: int((data.target==i).sum())
       for i, name in enumerate(data.target_names)})
```

```
Features (30): ['mean radius' 'mean texture' 'mean perimeter' 'mean area'
'mean smoothness' 'mean compactness' 'mean concavity'
'mean concave points' 'mean symmetry' 'mean fractal dimension'
'radius error' 'texture error' 'perimeter error' 'area error'
'smoothness error' 'compactness error' 'concavity error'
'concave points error' 'symmetry error' 'fractal dimension error'
'worst radius' 'worst texture' 'worst perimeter' 'worst area'
'worst smoothness' 'worst compactness' 'worst concavity'
'worst concave points' 'worst symmetry' 'worst fractal dimension']
Target names: ['malignant' 'benign']
X shape: (569, 30)
y distribution: {'malignant': 212, 'benign': 357}
```

```
In [2]: # 1) Стэкинг (StackingClassifier)
from sklearn.ensemble import StackingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.neighbors import KNeighborsClassifier

estimators = [
    ('logistic_regression', LogisticRegression(random_state=42, solver='liblinear')),
    ('random_forest', RandomForestClassifier(random_state=42)),
    ('gradient_boosting', GradientBoostingClassifier(random_state=42)),
    ('knn', KNeighborsClassifier()),
    ('svc', SVC(kernel='rbf', probability=True, random_state=42)),
]
stack = StackingClassifier(
    estimators=estimators,
    final_estimator=LogisticRegression(),
    cv=5
)
stack.fit(X_train, y_train)
print("Stacking acc:", stack.score(X_test, y_test))
```

```

# 2) Многослойный персептрон
from sklearn.neural_network import MLPClassifier

mlp = MLPClassifier(
    hidden_layer_sizes=(50,25),
    activation='relu',
    solver='adam',
    max_iter=300,
    random_state=42
)
mlp.fit(X_train, y_train)
print("MLP acc:", mlp.score(X_test, y_test))

```

Stacking acc: 0.9590643274853801  
MLP acc: 0.9122807017543859

```

In [3]: from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report
from gmdhpy.gmdh import (
    MultilayerGMDH,
    SequenceTypeSet,
    RefFunctionType,
    CriterionType
)

data = load_breast_cancer()
X_train, X_test, y_train, y_test = train_test_split(
    data.data, data.target,
    test_size=0.3, random_state=42, stratify=data.target
)

# Линейный GMDH (только linear)
model_lin = MultilayerGMDH(
    seq_type=SequenceTypeSet.sqModel,
    ref_functions=RefFunctionType.rfLinearCov,
    criterion_type=CriterionType.cmpTest,
    max_layer_count=10,
    normalize=True,
    print_debug=False,
    n_jobs=-1
)
model_lin.fit(X_train, y_train)
y_pred_lin_cont = model_lin.predict(X_test)
y_pred_lin = (y_pred_lin_cont > 0.5).astype(int)
print("Linear GMDH:")
print(" Accuracy:", accuracy_score(y_test, y_pred_lin))
print(classification_report(y_test, y_pred_lin, target_names=data.target_name

# Нелинейный GMDH (quadratic + cubic)
model_nonlin = MultilayerGMDH(
    seq_type=SequenceTypeSet.sqModel,
    ref_functions=(RefFunctionType.rfQuadratic, RefFunctionType.rfCubic),
    criterion_type=CriterionType.cmpTest,

```

```

    max_layer_count=10,
    normalize=True,
    print_debug=False,
    n_jobs=-1
)
model_nonlin.fit(X_train, y_train)
y_pred_nonlin_cont = model_nonlin.predict(X_test)
y_pred_nonlin = (y_pred_nonlin_cont > 0.5).astype(int)
print("Non-linear GMDH:")
print(" Accuracy:", accuracy_score(y_test, y_pred_nonlin))
print(classification_report(y_test, y_pred_nonlin, target_names=data.target_r

```

Linear GMDH:

Accuracy: 0.9532163742690059

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| malignant    | 1.00      | 0.88   | 0.93     | 64      |
| benign       | 0.93      | 1.00   | 0.96     | 107     |
| accuracy     |           |        | 0.95     | 171     |
| macro avg    | 0.97      | 0.94   | 0.95     | 171     |
| weighted avg | 0.96      | 0.95   | 0.95     | 171     |

Non-linear GMDH:

Accuracy: 0.9532163742690059

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| malignant    | 0.91      | 0.97   | 0.94     | 64      |
| benign       | 0.98      | 0.94   | 0.96     | 107     |
| accuracy     |           |        | 0.95     | 171     |
| macro avg    | 0.95      | 0.96   | 0.95     | 171     |
| weighted avg | 0.95      | 0.95   | 0.95     | 171     |

In [ ]: